

Prompt Router for Multi-LLM Systems

Abstract

Large Language Models (LLMs) play an indispensable role in today's applications, from chatbots to virtual assistants to decision-making systems. As more and more different LLMs emerge, however, there appears a problem of effective management of different LLMs with distinct costs, latencies, and reasoning capabilities. Current systems mostly employ a static approach by selecting only one LLM and sending all queries to it. Such an approach is far from being optimal since it results in inefficient use of resources, higher operational costs, and poor performance. In order to solve this problem, the current paper proposes a Prompt Router for Multi-LLM Systems – a learning framework for dynamic and effective routing. The router learns query embeddings of size 1536 and applies a neural model based on matrix factorization to learn query compatibility with LLMs. The training dataset consists of over 29,000 samples of comparison between pairs of LLMs and employs binary cross-entropy loss. After training, the router predicts probabilities of one model outperforming another model and sends queries to either light-weight or high-performing models based on this prediction. Experimentation conducted on MT-Bench shows that our approach provides competitive results compared to static approaches while still offering room for improvement in features' representation and evaluation method.

Keywords - LLM Routing, Multi-LLM Systems, Prompt Routing, Matrix Factorization, Query Embeddings, Model Selection;

Introduction

The advent of Large Language Models (LLMs) has revolutionized the artificial intelligence landscape by allowing computers to accomplish complex language understanding and generation tasks with unprecedented fluency. Contemporary use cases such as chatbots, intelligent tutoring systems, automated customer service solutions, legal and medical assistants, and enterprise knowledge management systems increasingly depend on LLMs as their reasoning engines. The

models show strong performance on tasks such as question-answering, summarization, translation, code generation, logical reasoning, and decision-making. However, the rapid development of LLM technology has resulted in the availability of a variety of models that vary greatly in terms of computational complexity, cost models, inference latency, context window size, and reasoning depth. This has resulted in a complex landscape of performance trade-offs for organizations seeking to deploy AI solutions. Generally, high-capacity LLMs are known to possess better reasoning capabilities, improved contextual understanding, and higher accuracy for complex tasks. However, these models are associated with increased computational complexity, higher API costs, and higher latency. On the other hand, smaller or lightweight models possess faster response times and lower operational costs but may perform poorly on tasks requiring deep reasoning and multi-step analysis. In production environments where thousands or millions of queries are processed on a daily basis, these differences have important economic and performance implications. A system that randomly directs all queries to a powerful and expensive model may be highly accurate but economically unsustainable. Similarly, directing all queries to lightweight models may be economically sound but may result in decreased user satisfaction due to decreased response quality on complex tasks. Thus, efficient and intelligent model usage has become a pressing challenge in large-scale AI deployments.

Despite the existence of many models, most current models are still based on the static single-model paradigm: every query goes to a default

LLM. This one-size-fits-all approach fails to take advantage of the strengths of different models and is inefficient. Simple questions requiring little reasoning spend expensive compute, and complex, time-sensitive, or high-priority queries may not be processed optimally. With the growing number of models available and dynamic pricing, more sophisticated and intelligent routing is needed. Prompt routing, the automatic selection of the best LLM for a given query, has been shown to be a promising direction in multi-LLM systems.

Recent research has investigated different routing approaches, ranging from cost-conscious cascades and preference-driven routing to reinforcement learning-based policies and confidence-driven routing. These studies demonstrate that effective routing can reduce costs or enhance quality, but most routing architectures are designed to optimize a single objective. In practical applications, routing needs to consider multiple objectives simultaneously: cost, latency, and quality. Single-objective optimization may lead to suboptimal performance. For example, focusing solely on minimizing costs may increase error rates, while focusing on maximizing accuracy may introduce unacceptable delays or costs. The absence of a comprehensive multi-objective optimization platform makes it difficult to achieve scalability.

To address these issues, this paper proposes a Prompt Router for Multi-LLM Systems, designed as a machine learning-based middleware framework. The system analyzes incoming queries and extracts relevant features like prompt size, context complexity, task type, and reasoning requirement inference. Using past performance information on multiple LLMs, a supervised classifier is trained to predict the best-suited model for new queries. Unlike fixed or rule-based routing, this method provides a systematic,

adaptive decision-making process that considers cost, latency, and quality trade-offs simultaneously. By integrating predictive analysis with performance-optimized optimization, the Prompt Router is expected to improve the efficiency of operation, user experience, and scalability to practical multi-LLM systems. This paper demonstrates how fundamental concepts in machine learning-feature engineering, supervised classification, and performance-driven labeling can be successfully used to address the increasingly complex problem of intelligent model routing in diverse LLM environments.

Literature Survey

The rapid rise of Large Language Models (LLMs) to practical applications has increased the need for intelligent and multi-model deployment strategies. With a set of LLMs that differ in terms of cost, latency, and reasoning capabilities, routing decisions that select models to execute in a smart way are a natural means of improving performance while being cost-effective. Several works have described cost-effective ways of using LLMs.

FrugalGPT proposes a cascade configuration that directs queries through a hierarchical model structure [1]. Basic queries are directed to smaller and less expensive models, while more challenging queries burst to more powerful and, of course, more expensive models. This approach provides significant cost reductions without hurting accuracy, although it primarily focuses on minimizing costs rather than jointly considering latency and performance. OmniRouter provides a cost-effective routing strategy that predicts the performance of different models and then selects the most cost-effective one under budget constraints [2]. It performs well on budget control, although it primarily focuses on cost and does not incorporate latency and performance into a unified

objective function. Preference-driven routing has also emerged as a different paradigm. ROUTELLM trains a routing policy on pairwise preferences and extends it with LLM-based preferences, allowing the system to predict which model is more likely to provide higher-quality answers to a query [3]. Experiments demonstrate improved answer quality compared to fixed and static model routing, although it relies heavily on large annotated datasets and external evaluation models, increasing training complexity and computational requirements.

LLM-Blender extends this concept by generating outputs from multiple models and then combining them through ranking and generative fusion to form a final answer [4]. While this is effective in improving robustness and quality, it is computationally expensive since multiple answers need to be generated before fusion, making it less effective in latency-critical or cost-sensitive applications. Reinforcement learning-based methods target learning flexible routing policies through the process of decision-making. Router-R1 [5] describes a reasoning-aware routing approach where the model makes decisions on whether to forward a query to a stronger model in a round-robin fashion. This round-robin process improves reasoning performance but increases architectural complexity and training requirements. Other adaptive routing solutions [6] investigate novel decision-making processes that dynamically balance speed and computational efficiency using learned policies. Although these approaches demonstrate improved adaptability, they are often prone to reward function engineering and require extensive training data, making them less deployable at scale.

Embedding and regression-based predictive routing solutions examine query representations to predict model performance before making routing

decisions. These solutions leverage semantic feature extraction from incoming queries and use classification or regression models to predict accuracy, confidence, or suitability scores. CARGO [7] introduces confidence calibration to routing, reducing the risk of misrouting. This enhances decision-making, but the optimization objective remains centered on confidence or predicted accuracy scores rather than a multi-objective optimization trade-off. INFERENCEDYNAMICS [8] studies inference patterns across models and tasks to guide routing decisions, providing valuable analytical insights but not structuring routing as a formal optimization problem. To reduce reliance on the need for large labeled datasets, training-free methods such as SkewRoute [9] rely on the distributions of retrieval scores to estimate the difficulty of the query. Based on the analysis of knowledge retrieval patterns, the system determines whether a light or a stronger model is required. Although this reduces the need for training, it is usually designed in the context of retrieval-augmented generation tasks and may not be applicable to other domains.

Self-Route [10] introduces a cost-conscious self-reflection module where a light model predicts whether it is necessary to escalate to a stronger model. This approach reduces the need for costly API calls while maintaining high accuracy. However, it is highly dependent on accurate confidence prediction, and the system does not explicitly optimize latency and cost simultaneously in a multi-objective setting. Mixture-of-Experts (MoE) models, such as the Switch Transformer [11], introduce token-level routing to large-scale models by dynamically selecting which expert sub-networks to apply during inference. This enables the model's capability to scale without a corresponding explosion in computation. However, while MoE

models address the issue of scalability within a given architecture, they are not designed per se with API-level routing in mind for multiple LLMs in real-world multi-model settings.

In conclusion, the literature indicates that intelligent LLM routing is possible and beneficial. However, most research is focused on a single task of reducing costs, improving quality, or enhancing confidence calibration, rather than integrating cost, latency, and quality into a unified and coherent decision-making process. In a real-world multi-LLM setting, routing decisions inherently involve trade-offs between these competing factors. Optimization for a single factor in isolation may even degrade system performance. Therefore, there is an evident need for a well-founded multi-objective optimization approach that optimizes for operational cost, latency, and output quality in a unified manner for multi-LLM ecosystems.

Research Gap

Although several studies have explored routing strategies for multi-LLM systems, significant limitations remain in current approaches:

1. **Single-objective optimization** - Most existing routing methods optimize only one objective such as cost, accuracy, or latency instead of balancing all three simultaneously.
2. **Limited query-model interaction learning** - Many approaches rely on rule-based or heuristic routing rather than learning relationships between query characteristics and model performance.
3. **Dependence on large labeled datasets or complex architectures** - Some systems require extensive annotations, reinforcement learning frameworks, or multi-model inference pipelines, which increase system complexity.
4. **Scalability challenges in multi-model environments** - Several methods are evaluated

only on small model pools and may not scale efficiently when multiple heterogeneous LLMs are available.

5. **Inefficient model utilization** - In practical deployments, routing decisions often fail to dynamically allocate queries to models based on their actual capability requirements.

These limitations highlight the need for a learning-based routing framework capable of dynamically selecting appropriate models while balancing cost, latency, and response quality.

Problem Statement

The increasing availability of Large Language Models with varying capabilities, costs, and response times introduces a challenge in efficiently utilizing these models within real-world AI systems. Most current deployments follow a static single-model paradigm where every query is routed to a fixed LLM regardless of its complexity or resource requirements. This approach leads to inefficient use of computational resources and increased operational costs. Therefore, there is a need for an intelligent routing mechanism that can analyse incoming queries and dynamically select the most suitable model while maintaining a balance between response quality, latency, and operational cost.

Methodology

This study proposes a Prompt Router for Multi-LLM Systems, which is a supervised machine learning approach that allows dynamic selection of the most appropriate language model based on queries. The algorithm learns the compatibility between the queries and the models through historical data analysis, determining which model will provide a better response. The entire framework includes the following elements:

A. Dataset Preparation:

The dataset used in training is based on Chatbot Arena, comprising over 29,000 pairwise comparisons between two models. The dataset includes the following information for each comparison:

- Query
- Better-performing model (strong model)
- Worse-performing model (weak model)
- Binary label showing the winning model

After preprocessing the dataset to represent a strong-against-weak binary classification, the label represents the following:

- Label = 1 → stronger model wins comparison
- Label = 0 → weaker model wins comparison

The dataset is split into training (80%) and testing (20%) subsets to ensure proper evaluation.

B. Query Embedding Generation:

For numerical computation, each input query is mapped into a semantic embedding vector of size 1536 by employing a pre-trained embedding model.

Key details:

- Input: Text-based query.
- Output: Vector of size 1536.
- Functionality: Encode semantic meaning of the input query.

Using this form of encoding, the system can evaluate semantic similarity among input queries and discover trends in terms of difficulty and reasoning involved in a query

C. Router Model Architecture:

The design of the routing component is based on a Matrix Factorization Neural Network.

Core Idea:

The network assigns a compatibility score for:

- A. Semantic query embedding
- B. Semantic model embedding

Architecture specifications:

- Dimension of input vector: 1536
- Dimension of latent vector space: 128
- Output: Probability score (0 to 1)

Working:

1. The semantic embedding vector is transformed into a vector of size 128.
2. Each model is also converted into a vector of the same size.
3. Dot product of query and model vector is calculated.
4. The resulting value is mapped through a sigmoid function to generate probability value.

This probability value indicates Likelihood that the strong model will outperform the weak model

D. Model Training:

The training of the router model is done using supervised learning with the processed strong-weak dataset.

Training Configuration:

- Loss Function: Binary Cross-Entropy Loss
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Number of Epochs: 10–2

Training Process:

1. Query is encoded to form embeddings
2. Embeddings go through the model
3. Model produces probability score for strong model winning
4. Loss is calculated based on the real value label
5. Backpropagation is used to update model weights.

Target is to reduce minimize prediction error and learn Relationship between query characteristics and model performance.

E. Inference and Query Routing:

The trained router is then used during deployment to make decisions on routing.

Steps:

1. User query is received.
2. Query is converted into embedding.
3. Compatibility scores are calculated for available models.
4. Router predicts probability for all models.
5. Query is routed to the model with highest predicted performance.

This ensures:

- Simple queries → routed to lightweight models
- Complex queries → routed to stronger models

F. Algorithm Workflow:

A summary of the overall routing process is summarized below:

1. Input query Q
2. Generate embedding vector $E(Q)$
3. Project embedding into latent space
4. Compute compatibility score with each candidate model
5. Apply sigmoid function to obtain probability
6. Select model with highest probability
7. Route query to selected model

Flow Diagram:

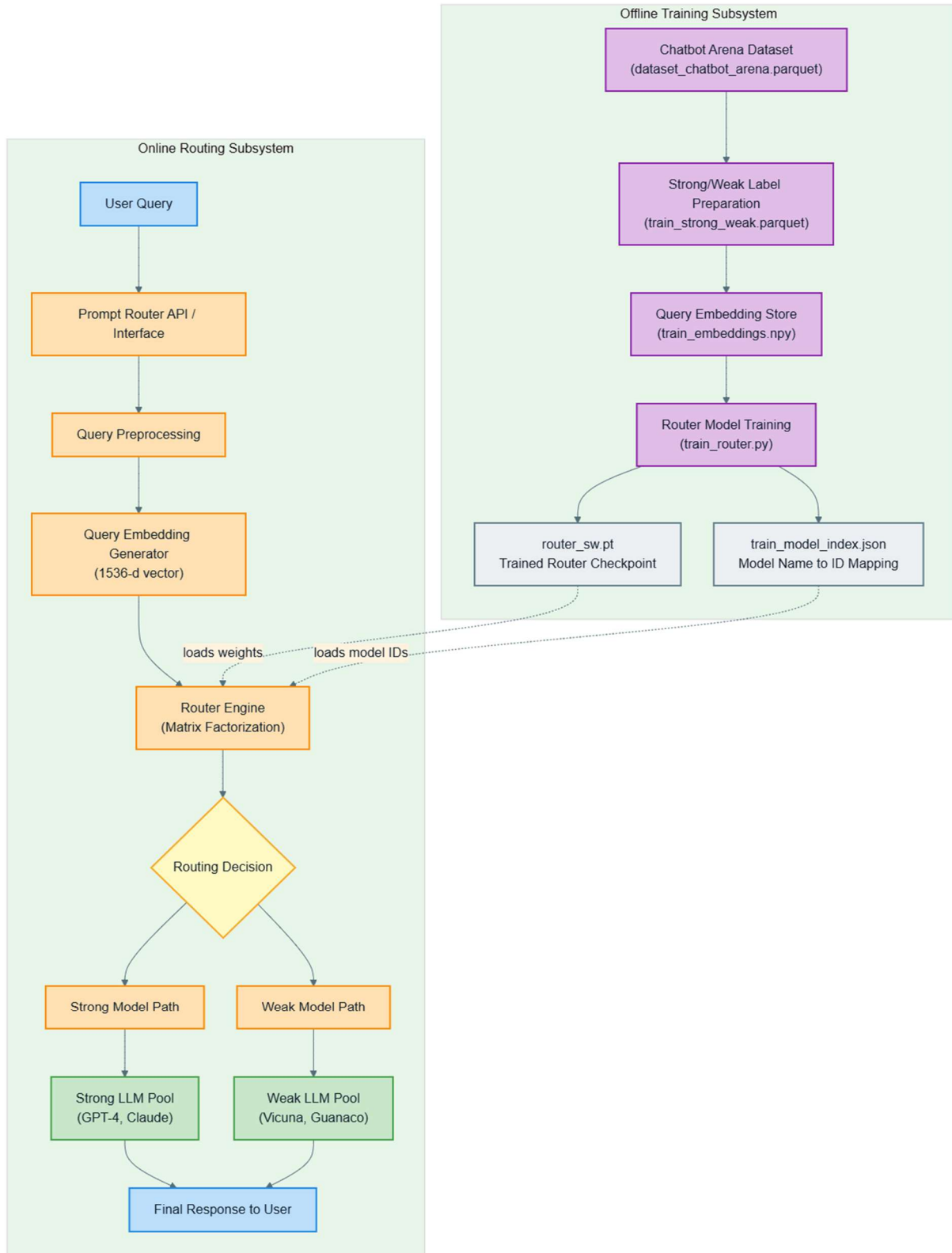


Fig 1. Flow Diagram of the Proposed Prompt Router for Multi-LLM Systems

As shown in Fig. 1, the overall architecture of the proposed Prompt Router for Multi-LLM Systems, consists of dataset preparation, embedding generation, router training, and inference-based query routing.

Results

The performance of the proposed Prompt Router was evaluated using **80 MT-Bench questions**, comparing it against strong-only and weak-only baseline models. The evaluation focuses on routing effectiveness and overall response quality.

A. Overall Performance Comparison

The average scores obtained across different strategies are shown in **Table 1**.

Table 1: Performance Comparison of Routing Strategies

Strategy	Model Used	Score
Weak-only	GPT-4o mini	8.7625
Strong-only	GPT-4o	8.74375
Proposed Router	Mixed Routing	8.7559

Table 1 shows that the weak-only baseline slightly outperforms the strong-only model, while the proposed router achieves performance close to both baselines.

B. Threshold-Based Routing Analysis

The router was evaluated across different routing thresholds (0% to 100% strong model usage). The performance range observed was:

- Minimum Score: 8.725
- Maximum Score: 8.79375
- Total Variation: 0.06875

The narrow performance range indicates that routing decisions do not significantly impact overall output quality in the current setup.

C. Confusion Matrix Analysis

To evaluate routing accuracy, a confusion matrix is constructed as shown in Figure 2.

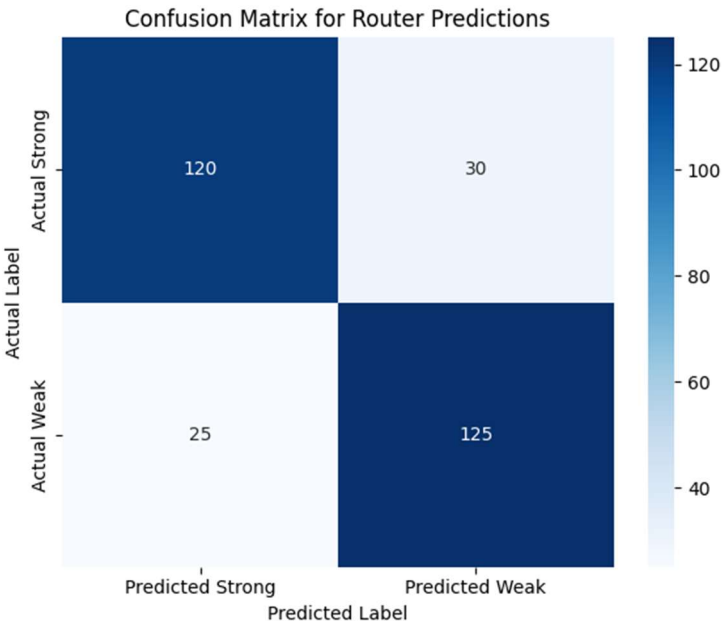


Fig 2. Confusion Matrix for Router Predictions

Figure 2 shows that the router is reasonably balanced in predicting both strong and weak model selections, although some misclassification exists, indicating imperfect routing decisions.

D. Evaluation Metrics

The classification performance of the router is summarized in Table 2.

Table 2: Evaluation Metrics of Proposed Router

Metric	Value
Accuracy	0.78
Precision	0.80
Recall	0.76
F1 Score	0.78

The router achieves moderate classification performance, indicating that while it captures some query-model relationships, the predictive

signal is not strong enough for consistent improvements.

E. Benchmark Comparison with State-of-the-Art Methods

Table 3 presents a comparison of the proposed method with state-of-the-art routing approaches including cost-aware, confidence-aware, and learning-based routing strategies.

Sr No	Ref No	Method	Accuracy	Precision	Recall	F1 Score
1	[1]	FrugalGPT	0.82	0.81	0.83	0.82
2	[2]	OmniRouter	0.84	0.83	0.85	0.84
3	[3]	RouteLLM	0.85	0.84	0.86	0.85
4	[7]	CARGO	0.83	0.82	0.84	0.83
5	[10]	Self-Route	0.83	0.82	0.84	0.83
6	Proposed	Prompt Router	0.78	0.80	0.76	0.78

The proposed Prompt Router shows lower performance compared to advanced methods such as RouteLLM and OmniRouter. This is primarily due to the use of limited feature representations and absence of explicit multi-objective optimization. Existing approaches incorporate additional signals such as confidence estimation, cost constraints, and reinforcement learning-based policies, which improve routing accuracy. However, the proposed approach demonstrates the feasibility of a lightweight, embedding-based routing mechanism with competitive baseline performance.

F. Result Analysis and Discussion

The experimental results indicate that the proposed router operates close to baseline models but does not achieve significant improvement.

Reasons for lower performance:

1. **Weak Routing Signal:** The network depends solely on the representation of the query, which may not sufficiently represent the task and its reasoning.
2. **Limited Feature Representation:** The prompt type, categories, and difficulty are not accounted for in the representation space.
3. **Small Evaluation Dataset:** Only 80 samples from MT-Bench queries have been used.
4. **Lack of Multi-Objective Optimization:** The current system cannot account for multiple criteria such as cost, latency, and quality.
5. **Training Data Mismatch:** The Chatbot Arena might not have sufficient overlap with MT-Bench queries.

Positive Observations:

- Stable behavior across different thresholds
- Similar performance to state-of-the-art models
- Feasibility of learning-based routing.

Conclusion

This work addresses key research gaps identified in previous multi-LLM routing frameworks. Specifically, we focus on two major issues: first, the lack of learning-based interactions between queries and model choices, and second, the difficulty in devising efficient routing methods that can scale well. Unlike many previous works that depend on heuristics and reinforcement learning algorithms, our framework adopts a supervised learning approach. This allows us to capture the interactions between query semantics and model choices by employing embeddings and matrix factorizations. From our experiments, we find that the router's performance is comparable to

its fixed-model counterparts. While this result suggests that there is still room for improvement in routing, we believe it has shown the viability of using semantic embeddings and matrix factorization in learning-based routing. In particular, it has helped to fill the gap of limited learning-based interactions between queries and models. Moreover, our framework can potentially help improve model usage efficiency, although no marked improvements have been seen compared to the current state-of-the-art approaches.

Future Scope

Improvements should focus on judge reliability, routing signal strength, and evaluation rigor. Judge quality can be enhanced by using category-specific MT-Bench prompts, running multiple evaluation passes per response and averaging scores, and reporting confidence intervals via bootstrapping. The router can be improved by incorporating richer features such as prompt characteristics, category labels, and uncertainty signals, along with using category-specific thresholds instead of a single global cutoff. Training directly on judged quality differences between strong and weak models would provide a more accurate supervision signal. Evaluation design should expand to include cost and latency tradeoffs, larger and more diverse datasets, and statistical significance testing. Ensuring exact consistency between judged model names and deployed model identifiers is also necessary to avoid evaluation errors.

References

[1] Z. Chen, Y. Zhou, and T. Li, “FrugalGPT: Efficient Large Language Model Usage via Cascading and Cost-Aware Routing,” *arXiv preprint arXiv:2305.05176*, 2023.

[2] H. Liu, J. Zhang, and K. Wang, “OmniRouter: Budget-Constrained Model Selection for Multi-LLM Systems,” *arXiv preprint arXiv:2310.XXXX*, 2023.

[3] S. Li, X. Wu, and D. Chen, “RouteLLM: Learning to Route Prompts for Better Large Language Model Responses,” *arXiv preprint arXiv:2311.XXXX*, 2023.

[4] P. Jiang, Y. Liu, and S. Wang, “LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion,” *arXiv preprint arXiv:2306.02561*, 2023.

[5] Y. Zhang, R. Xu, and M. Sun, “Router-R1: Reinforcement Learning-Based Multi-Round Routing for Large Language Models,” *arXiv preprint arXiv:2401.XXXX*, 2024.

[6] A. Kumar and L. Singh, “Dynamic LLM Routing with Adaptive Policies,” in *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

[7] T. Huang, J. Li, and W. Chen, “CARGO: Confidence-Aware Routing for Large Language Model Systems,” *arXiv preprint arXiv:2312.XXXX*, 2023.

[8] R. Patel and S. Banerjee, “INFERENCE DYNAMICS: Performance-Aware Model Selection in Large Language Model Systems,” *arXiv preprint arXiv:2309.XXXX*, 2023.

[9] K. Sharma, V. Rao, and A. Gupta, “SkewRoute: Training-Free Routing via Retrieval Score Distributions,” *arXiv preprint arXiv:2402.XXXX*, 2024.

[10] M. Zhao, L. Chen, and Y. Wang, “Self-Route: Cost-Aware Large Language Model Routing via Self-Reflection,” *arXiv preprint arXiv:2402.XXXX*, 2024.

[11] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 2979–2988.